

# Analysis: Beaming With Joy

## Beaming With Joy: Analysis

### Small dataset

There are a lot of ways to solve the Small dataset. The most straightforward way is to use dynamic programming. Since the input contains no mirrors, each laser beam covers a set of non-wall cells that are either horizontally or vertically consecutive. Let us process the columns from left to right. For each row, we will keep track of its current status, which must be one of the following:

1. It has an incoming beam, which originated in this row in a previously processed column and was not yet blocked by a wall.
2. It needs a beam — that is, there is at least one cell in this row from a previously processed column that has not yet been covered by a beam, but could be (i.e. there is not a wall in the way).
3. It does not have nor it can receive a beam, because this would destroy a beam shooter pointing vertically that we saw in this row in a previously processed column.
4. None of the above. This means that all previous consecutive non-wall cells, if any, are covered with vertical beams, so we can shoot horizontally on this row, but there is no obligation yet to do so.

There are some ways to refine this to fewer combinations, but it's not necessary. Then, we can try all combinations of orientations for the beam shooters in the current column and see if that produces a valid status for the next column (there are more efficient ways to do this part, but this will do). This requires a bit of casework: for each row, we have to combine its incoming status with the current cell. The current cell can be one of five things: a covered or uncovered empty cell, a vertical or horizontal beam shooter, or a wall. Then, since there are 4 possible states for each row, the domain of our recursive function is of size  $4^R \times C$  and the calculation of each value in that domain requires checking up to  $2^R$  combinations of beam shooters' orientations. Each such check requires a linear pass over the  $R$  cells, making the overall time complexity bounded by  $O(8^R \times C \times R)$ . It's not hard to come up with even smaller bounds, given that many combinations of statuses are actually impossible.

There is also a greedy Small solution that relies only on the absence of mirrors:

1. If a beam shooter can possibly destroy another, then point it the other way. If both directions would destroy another beam shooter, then the case is impossible.
2. If there are uncovered cells that can only be covered by a single non-yet-fixed shooter, then point that shooter in its direction. If any remaining cells are impossible to cover (because any beam shooters that may have pointed at it had to be fixed in the other direction), then the case is impossible.
3. Make all beam shooters that are not fixed yet shoot horizontally (or all vertically, which also works).

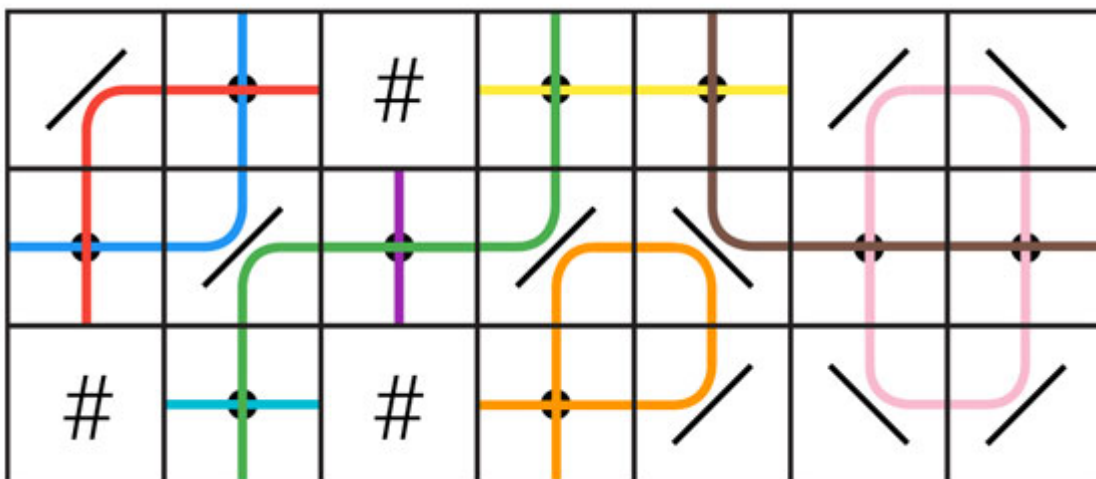
This works because if any cell remains uncovered after the first two steps, it can only be because there are exactly two remaining non-fixed beam shooters potentially pointing at it: one horizontally, and one vertically. If there had been two or more beam shooters pointing at it in the same direction, we would have already fixed all of them in step 1; if there had been only one beam shooter in total, we would have fixed it in step 2. Then, choosing the same direction for all

beam shooters ensures that for all those cells, (exactly) one of the beam shooters is pointed in the correct direction to cover that cell.

## Large dataset

The second solution presented above for the Small dataset hints at a possible generalization that solves the Large dataset.

In the Small, each cell is at an intersection of a horizontal line of consecutive non-wall cells, and a vertical line of consecutive non-wall cells. These lines of cells run between walls, between opposite grid boundaries, or between one wall and one grid boundary. Each of these lines of cells is characterized by having either no beam shooter (which forces each cell to be covered from the other direction in step 2), one beam shooter (which leaves the cell on hold until step 3), or more than one (which forces all beam shooters on the line to point in the direction opposite to the line of cells in step 1). In the Large, something similar happens, but instead of simple lines of cells, we have the more complicated notion of *paths*. A path is a set of pairs  $(c, d)$  where  $c$  is any cell that is empty or contains a beam shooter and  $d$  is either horizontal or vertical. A pair  $(c, d)$  and a different pair  $(c', d')$  are part of the same path if and only if a beam shooter placed on cell  $c$  and pointing in direction  $d$  would produce a beam that, ignoring all other beam shooters, would pass through cell  $c'$  in direction  $d'$ . As an example, the following picture illustrates a grid with 9 paths.



Notice that the red and blue paths pass through the same set of empty cells, although the (cell, direction) pairs that make up those paths are different. Also notice there is an orange path that passes through the same cell twice; that is, the path contains the pair  $(c, \text{horizontal})$  and the pair  $(c, \text{vertical})$  where  $c$  is the rightmost empty cell in the bottom row. Also notice how some paths may contain a single pair, like the turquoise, purple, and brown paths in the picture. Finally, some paths loop onto themselves, like the pink path in the picture, while other paths start and end at a wall or grid boundary. We call the former *loop paths*.

Paths that go through a number of beam shooters other than one have similar immediate consequences to lines of consecutive non-wall cells in the Small dataset. For convenience, let us define the opposite function  $o$  given by  $o(\text{horizontal}) = \text{vertical}$  and  $o(\text{vertical}) = \text{horizontal}$ .

1. If a path contains two pairs  $(c, d)$  and  $(c, o(d))$  where  $c$  is a cell that contains a beam shooter, the case is impossible, because that means that beam shooter will destroy itself when pointed in either direction.
2. If a loop path contains a pair  $(c, d)$  where  $c$  is a cell that contains a beam shooter, the beam shooter must be fixed in direction  $o(d)$ .
3. If a path contains two or more pairs  $(c_i, d_i)$  where  $c_i$  contains a beam shooter, then beam shooter  $c_i$  must be fixed in direction  $o(d_i)$ .

4. If there is no beam shooter on a path that contains two pairs  $(c, d)$  and  $(c, o(d))$  where  $c$  is an empty cell, the case is impossible (as  $c$  cannot be covered).
5. If there is no beam shooter on a path, then for each pair  $(c_i, d_i)$  it contains: let  $p$  be the path containing  $(c_i, o(d_i))$ . If  $p$  contains a single pair  $(c, d)$  where  $c$  contains a beam shooter, then fix that beam shooter in direction  $d$ . If  $p$  contains a number of beam shooters other than one, the case is impossible.

If a beam shooter is required to be fixed in two different directions, by the same or different steps, the case is impossible. This is the generalization of steps 1 and 2 in the Small greedy solution. After applying these, some cells may be left uncovered. As before, those cells are always in the intersection of two different paths, and those paths contain a single pair where the cell contains a beam shooter each. Unfortunately, the way to set the remaining beam shooters so that every cell is covered is not as simple as step 3 for the Small. This might start to look like a (non-bipartite) matching problem, but it is not one!

In the Small, after all the cases that directly fixed the direction of some beam shooter, each empty cell that remained uncovered could possibly be covered by two beam shooters, one on each direction. The same is true in this case: for each empty cell  $c$  that remains uncovered there are two different paths that contain the pairs  $(c, \text{horizontal})$  and  $(c, \text{vertical})$ , and each of them passes through exactly one pair that contains a non-yet fixed beam shooter. Say those two pairs containing shooters are  $(s_1, d_1)$  and  $(s_2, d_2)$ . Then, covering cell  $c$  requires us to either set  $s_1$  in direction  $d_1$  or  $s_2$  in direction  $d_2$ , or both. If it happens to be  $s_1 = s_2$ , then it must be  $d_1 \neq d_2$ , and the requirement is fulfilled by any assignment.

If we assign logical variables to the shooters with the logical values true and false corresponding to the two directions, each of these restrictions is a [disjunction](#) of two [literals](#). Making all such disjunctions simultaneously true is, then, making their [conjunction](#) true. Finding a truth assignment to variables to make a conjunction of disjunctions of up to two literals each true is a problem known as the [2-satisfiability problem](#) or 2SAT. We can use known 2SAT algorithms to solve that problem and get an assignment for the variables, and then translate that assignment back into an assignment of directions for the beam shooters.

The greedy Small-only solution we presented is even simpler to prove correct under this logic model: without mirrors, every disjunction has exactly one literal negated (because the cell requirement is for one beam shooter to be horizontal and/or some other beam shooter to be vertical), so assigning all variables a true value makes all disjunctions true thanks to the non-negated literal, and also assigning them all a false value makes all disjunctions true thanks to the negated literal.

This analysis is written in the order in which one might reason this problem step by step, without any huge leap at any single step. However, for some people it might be faster to have an aha moment and notice that each cell yields a requirement on up to two beam shooters directly: cells without shooters imply a requirement on up to two shooters, one of each direction, to be true. If there is only one of those, that can be encoded as a single-literal disjunction, or made it a two-literal disjunction by simply taking literal  $L$  and writing  $(L \vee L)$ . Then, beam shooters that possibly point at another beam shooter yield another single-literal disjunction forcing them to point the other way. This makes the first pass unnecessary and encodes all requirements into the 2SAT instance, making the solution more concise.

This problem lends itself to many greedy heuristics plus some kind of brute force or backtracking. Notice that 2SAT itself can be solved in polynomial time by some backtrackings and by a graph-theoretical algorithm that has many greedy decisions underneath (both approaches are mentioned in the Wikipedia article). So, many algorithms that don't explicitly use an algorithm for the 2SAT problem are actually correct because they are basically doing the same thing without going through the modeling.

# Analysis: Fresh Chocolate

## Fresh Chocolate: Analysis

The main initial observation to have is that the group sizes are only important [modulo  \$P\$](#) . Then, we can just take the smallest possible equivalent size for each group, and further assume that all of them are within the range  $[1, P]$ . Moreover, you may find it easier to map them all to the range  $[0, P-1]$ , using the modulo operation, as long as you don't mind working with 0, which is not a real group size.

We can see now that a test case with  $P=2$  can be described by 2 integers: the number of groups with odd size, and the number of groups with even size. Similarly, each test case can be described by a tuple of exactly  $P$  integers  $a_0, a_1, \dots, a_{P-1}$ , where  $a_i$  is the number of groups of size equal to  $i$  modulo  $P$ .

The groups counted in  $a_0$ , i.e., those with a group size multiple of  $P$ , are the simplest. No matter when they are brought in for the tour, the number of leftovers after they are given chocolate is the same as before. Therefore, their position won't change how many of the other groups get all fresh chocolate. This implies that it's always optimal to greedily choose a position for them where they get all fresh chocolate, and that we can accomplish that by putting them all at the beginning. Since the starting group gets fresh chocolate, a stream of groups with sizes that are multiples of  $P$  will all get fresh chocolate and will leave no leftovers for the next group. That is, we can solve the problem disregarding  $a_0$ , and then add  $a_0$  to the result. After this simplification, there are only  $P-1$  numbers left to consider.

### Small dataset

For the Small dataset, there are only two possible values for  $P$ , so we can consider them separately.

For  $P=2$ , there is only one number to consider:  $a_1$ . Since all groups are equivalent, there are no decisions to be made. All odd-sized groups will alternate between getting all fresh chocolate and getting a leftover piece. Since the last group will get fresh chocolate if there is an odd number of such groups, the number of groups that get fresh chocolate is  $\text{ceiling}(a_1 / 2)$ .

For  $P=3$ , there are two numbers to consider:  $a_1$  and  $a_2$ . Intuitively, we should pair each 1-person group with a 2-person group to go back to having no leftovers as soon as possible, and that's indeed optimal: start with an alternation of  $\min(a_1, a_2)$  pairs, where we get  $\min(a_1, a_2)$  added to our result, and then add pairs, of which  $\min(a_1, a_2)$  will get all fresh chocolate. Then add all the remaining  $M = |a_1 - a_2|$  groups (all of size 1 or 2 modulo  $P$ , depending on which type had more to begin with). Of those,  $\text{ceiling}(M / 3)$  will get all fresh chocolate. This last calculation is similar to what we saw for odd-sized groups for  $P = 2$ . We will prove the optimality of this strategy below.

### Large dataset

The remaining  $P=4$  case is a bit more complicated than just combining our insights from the  $P=2$  and  $P=3$  cases. To formalize our discussion, and to prove the correctness of our Small algorithm, we will introduce some names. Let us call a group fresh if it is given all fresh

chocolate, and not fresh if it gets at least one leftover piece. Also, we will call a group with a number of people equal to  $k$  modulo  $P$  a " $k$ -group".

Given some fixed arrival order, let us partition it into blocks, where each block is an interval of consecutive groups that starts with a fresh group and doesn't contain any other fresh groups. That is, we start a new block right before each fresh group. The problem is now equivalent to finding the order that maximizes the number of blocks. A group is fresh if and only if the sum of all people in the groups that preceded it is a multiple of  $P$ . This implies that reordering within a block won't make any other blocks invalid, so it won't make a solution worse (although it could improve it by partitioning a block into more than one block). Also notice that, with the possible exception of the last block, reordering blocks also doesn't alter the optimality of a solution.

Suppose we have an optimal ordering of the groups, disregarding 0-groups as we mentioned above. First, we can see that if a block contains a  $k$ -group and a different  $(P-k)$ -group, then it only contains those two: otherwise, we can reorder the block putting the two named groups first, and since the sum of people of the two groups is a multiple of  $P$ , that partitions the block further, which means the original solution is not optimal. Second, let us show that it is always optimal to pair a  $k$ -group with a  $(P-k)$ -group into a block. Assume an optimal order with a maximal number of blocks consisting of a  $k$ -group and a  $(P-k)$ -group. Then, suppose there is a  $k$ -group in a block A with no  $(P-k)$ -group and a  $(P-k)$ -group in a block B with no  $k$ -group. We can build another solution by making a new block C consisting of the  $k$ -group from block A and the  $(P-k)$ -group from block B, and another block D consisting of the union of the remaining elements of blocks A and B. If D doesn't sum up to a multiple of  $P$ , that implies that either A or B didn't to begin with, so we can just place D at the end in place of the one that didn't. C can be placed anywhere in the solution. This makes a solution with an additional pair that is also optimal, which contradicts the assumption that a  $k$ -group and a  $(P-k)$ -group existed in separate blocks. This proves that, for  $P=3$ , it is always optimal to pair 1-groups and 2-groups as we explained above.

For  $P=4$ , the consequence of the above theorem is that we should pair 2-groups with themselves as much as possible and 1-groups with 3-groups as much as possible. This leaves at most one 2-group left, and possibly some 1-groups or 3-groups left over, but not both. Since we need four 1-groups or four 3-groups to form a non-ending block, and we can use a 2-group to form a non-ending block with only two additional groups of either type, it is always optimal to place the 2-group first, and then whatever 1-groups or 3-groups may be left. Overall, the solution for  $P=4$  is  $a_0$  as usual (singleton blocks of 0-groups), plus  $\text{floor}(a_2 / 2)$  (blocks of two 2-groups), plus  $\min(a_1, a_3)$  (blocks of a 1-group and a 3-group), plus  $\text{ceiling}((2 \times (a_2 \bmod 2) + |a_1 - a_3|) / 4)$  (the leftover blocks at the end).

Notice that even though the formality of this analysis may be daunting, it is reasonable to arrive at the solution by intuition, and the code is really short. If you have a candidate solution but you find it hard to prove, you can always compare it against a brute force solution for small values of  $N$  to get a little extra assurance. Or there is also ...

## A dynamic programming solution

The insight that a case is represented by a  $P$ -uple is enough to enable a standard dynamic programming solution for this problem. Use  $P$ -uples and an additional integer with the number of leftovers as current state, and recursively try which type of group to place next (there are only  $P$  alternatives). Memoizing this recursion is fast enough, as the domain is just the product of all integers in the initial tuple, times  $P$ . When the group types' sizes are as close to one another as possible, we get the largest possible domain size, which is  $(N/P)^P \times P$ . Considering the additional iteration over  $P$  possibilities to compute each value, the overall time complexity of this approach is  $O((N/P)^P \times P^2)$ . Even for the largest case, this is almost instant in a fast language, and gives plenty of wiggle room to memoize with dictionaries and use slower languages. A purposefully non-optimized implementation in Python takes at most half a second to solve a case in a modern machine, so it finishes 100 cases with a lot of time to spare. Moreover, just

noticing that groups of size multiple of  $P$  can optimally be placed first makes the effective value of  $P$  be one less, which greatly improves speed and makes the solution solve every test case instantly.

# Analysis: Roller Coaster Scheduling

## Roller Coaster Scheduling: Analysis

### Small dataset

There are various ways to solve the Small dataset; for instance, it can be reduced to a [flow problem](#). It is also possible to find a greedy strategy or even a formula: the number of rides needed is  $\max(\text{number of tickets for customer 1}, \text{number of tickets for customer 2}, \text{number of tickets with seat 1})$ , and the number of promotions needed is  $\max(0, (\text{maximum among counts of } i\text{-th seat among all tickets}) - \text{number of rides needed})$ . However, these strategies are tricky to prove. The rest of our analysis will shed some light on why they work.

### Large dataset

To solve the Large dataset, we can start with observations similar to those in the Small case: the number of tickets held by any one person is a lower bound on the number of total rides, and so is the number of tickets for position 1. This is because the set of tickets held by any one person, and the set of tickets for position 1, share the property that no two tickets in the set can be honored in the same ride, even with promotions. An extension of this is that for the set of all tickets for positions 1, 2, ..., K, at most K of them can be honored by a single ride (possibly using promotions). This means that, if  $S_K$  is the number of tickets for positions up to and including K,  $\text{ceil}(S_K / K)$  is also a lower bound on the final number of rides.

It is not difficult to see this when there is only one ticket per customer: if the maximum of all those lower bounds is R, for as long as there is some position P such that there are more than R tickets for position P, we can promote any ticket for position P to some previous position that has less than R tickets assigned, which is guaranteed to exist due to the  $R \geq \text{ceiling}(S_P / P)$ . After that, no position is assigned more than R tickets. Since there are no repeated customers, we can just grab one ticket for each position that has tickets remaining and assign them to a ride until there are no more left. This will yield an assignment with exactly R rides, and we proved above that there can't be less than R.

When there are customers holding more than one ticket, our greedy assignment in the last step above might fail. We can still prove that there is an assignment that works with a bit of mathematical modeling. Consider a fixed ride plan. Let us define the ride plan matrix as a square matrix of side  $S = \max(\mathbf{N}, \mathbf{C})$ . The first  $\mathbf{C}$  rows represent customers and the first  $\mathbf{N}$  columns represent positions. The remaining rows or columns represent fake customers or positions, whose role will be clear in a moment. For each ride in the plan, we construct a one-to-one assignment of customers and positions. Customers that participate in the ride are assigned to their position on it. Customers that do not are assigned to empty positions or fake positions. If there are more positions than customers, each empty position is assigned a fake customer. Then, the value of a given cell of the ride plan matrix is the number of times the represented customer was assigned to the represented position. Notice that the value is an upper bound on the number of times a customer actually rode in the position, but not the exact number.

Notice that for any ride plan consisting of R rides, its ride plan matrix will have rows and columns that sum up to R. This is because for each ride, there is exactly one cell per row and one cell per column that gets a 1 added to it. The most interesting realization is that we can go the other way: for any matrix M such that all its rows and columns add up to R, there is a ride plan consisting of R rides such that M is a ride plan matrix for it. The proof is a simple variation

on the [Birkhoff-von Neumann theorem](#), which implies a matrix with that property can be expressed as a sum of [permutation matrices](#), and each permutation matrix corresponds to a possible ride.

A given set of tickets can also be represented by a matrix of side  $S$  by having a cell contain the number of tickets a given customer holds for a given position. Let us say that a matrix  $M$  is less than or equal to another matrix  $M'$ , if and only if the cell at row  $i$  and column  $j$  in  $M$  is less than or equal to that the value of the cell at row  $i$  and column  $j$  in  $M'$ . After promotions, we need the matrix representing the tickets to be less than or equal to the ride plan matrix.

Notice now that the greedy promotion algorithm presented in the second paragraph of this section, actually yields a ticket matrix such that no row or column exceeds the established lower bound  $R$ , even if there is more than one ticket per customer. The proof is exactly the proof above. What was missing before was a way to know that set of tickets could be turned into an actual ride plan in the case where there is more than one ticket per customer. We now have such a way, which means the originally naive solution is actually a full solution for the problem. Moreover, since we need to report the minimum number of promotions and not the promotions themselves, we can just add  $S_P - R$  for each position  $P$  such that  $S_P > R$ , and we are done.



# Analysis: Shoot the Turrets

## Shoot the Turrets: Analysis

### Small dataset

The first thing to notice is that, if a given soldier  $s$  is not the first to perform her task, there is no reason for  $s$  to move at all until all soldiers that are to shoot a turret before  $s$  have finished.

Let us call the output a strategy: a pairing between some soldiers and all turrets, in a specific order. Given a fixed strategy, we can check if it's viable with one [BFS](#) per soldier, in order. We stop with the first soldier that can't reach its assigned turret, if any, and report the number shot so far. Any strategy that shoots a maximum number of turrets can be extended to a pairing of all soldiers to turrets, so this covers all possibilities. Besides the usual empty spaces and walls, we need to consider some empty spaces enterable but not exitable: the spaces that are in the line of sight of turrets that are to be destroyed by a soldier after the current one. Each check takes time linear in the size of the map, so  $O(RC)$  per soldier.

Unfortunately, there can be too many strategies to brute-force them. There can be up to  $10!$  possible pairings and  $10!$  orderings, which means  $10!^2$  strategies, which is way too big, so we need to do better. We will use precomputing to optimize the check and dynamic programming to optimize the strategy generation.

Let  $S$  be the number of soldiers and  $T$  the number of turrets. To make the check faster, we can compute, for each soldier and each possible subset of still-alive turrets, the set of reachable turrets to shoot. That requires at most  $2^T \times S$  BFSes like the ones mentioned above, for a total complexity of  $O(2^T \times SRC)$ , which is fine for the Small limits.

Now on to the dynamic programming: consider the function  $f(s, t)$  which finds a strategy that shoots a maximum number of turrets for a given set of remaining soldiers  $s$  and a given set of remaining turrets  $t$ , or decides that there isn't one. Each of  $s$  and  $t$  can be represented with a binary string of up to  $S$  and  $T$  digits, respectively. The domain of the function, then, is of size up to  $2^{S+T}$ . To compute the function, we can check every soldier against every reachable turret. Since that list is precomputed, this takes at most  $S$  iterations over lists of length up to  $T$ . The overall computation of  $f$ , if memoized, has a complexity of  $O(2^{S+T} \times ST)$ , which again, finishes comfortably in time under the Small limits.

### Large dataset

For the Large dataset, anything with exponential complexity seems doomed to fail, so we need to go a different route. We will reuse the the BFS idea (to check out whether turrets are reachable) from the Small, but there are a lot of additional insights.

Let us build a [bipartite graph](#)  $G$  with a node for each soldier and turret.  $G$  has an edge from soldier  $s$  to turret  $t$  if and only if soldier  $s$  can destroy turret  $t$  after all other turrets have been destroyed. A single BFS starting from each soldier can build this graph. We state the following: destroying  $R$  turrets is possible if and only if there is a [matching](#) in  $G$  that covers  $R$  turrets. The only if part is trivial, as the conditions for edges in the graph are a relaxation of the conditions for the pairing we need to construct as solution. We concentrate on proving the if part. Moreover, we provide here a computably constructive proof, for which there is an efficient enough algorithm, meaning the proof is also an algorithm that solves the problem.

First, if  $G$  is empty, the problem is solvable trivially by vacuity. For non-empty  $G$ , we can use the [Ford-Fulkerson](#) algorithm to find an initial maximum matching  $M$  of  $G$  of size  $R$ . We will further consider only the soldiers present in  $M$  and ignore the others. From now on, we refer only to soldiers matched by  $M$ , and we remove the unmatched soldier nodes from  $G$ . Let us define the graph  $G'$  with the same nodes as  $G$ , but an edge between soldier  $s$  and turret  $t$  only exists in  $G'$  if  $s$  can destroy  $t$  with the other turrets active. Clearly,  $G'$  is a subgraph of  $G$ . The [outdegree](#) of each soldier node in  $M$  is at least 1. If an edge  $(s, t)$  in  $G$  does not exist in  $G'$ , it is because some other turret  $t'$  is reachable by  $s$  and blocking the path to  $t$ . But then,  $(s, t')$  is in  $G'$ . Therefore, the outdegree of each soldier not in  $G'$  is at least 1.

Consider the graph  $H$  that is the union of  $G'$  plus the edges in  $M$  reversed. If  $H$  contains an edge  $(s, t')$  where  $t'$  is a turret node not covered by  $M$ , let  $M'$  be equal to  $M$ , but replacing  $(s, t)$  by  $(s, t')$ , where  $(s, t)$  was the edge covering  $s$  present in  $M$ . Of course,  $M'$  is a maximum matching of  $G$  of the same size as  $M$ . If there is no such edge in  $H$ , do the following: since  $H$  is a supergraph of  $G'$ , the outdegree of each soldier node in  $H$  is at least 1. The inclusion of the reversed edges of  $M$  in  $H$  means the outdegree of all turret nodes matched in  $M$  in  $H$  is at least 1. Therefore, starting at any soldier and moving through edges in  $H$ , we will always encounter nodes with outdegree 1 of soldiers and turrets covered by  $M$ , and eventually find a cycle of them. Let us call this cycle  $C$ . Notice that the edges going out of turrets in  $H$  are only the reversed edges from  $M$ , so  $C$  is necessarily an alternation of reversed edges from  $M$  and edges from  $G'$ . Consider a new matching  $M'$  of  $G$  consisting of the edges of  $M$  whose reverse is not in  $C$ , plus the edges in  $C$  whose reverse is not in  $M$ . That is,  $M'$  is  $M$  but exchanging the edges present in  $C$  in some direction.  $M'$  in this case is also a matching of  $G$  of the same size as  $M$ . If the cycle is of length 2, then  $M'$  ends up being exactly the same as  $M$ .

In both cases, we constructed a new matching  $M'$  of the same size as  $M$  with the additional property that at least one edge of  $M'$  is present in  $G'$ . Therefore, there are some edges in the matching that represent a number  $A > 1$  of actions that we can take now. So, we can just take those  $A$  actions, remove all  $A$  used soldier nodes and  $A$  destroyed turret nodes, and we are left with a smaller graph which also has a maximum matching of size  $R - A$  (whatever is left of  $M'$ ). Rinse and repeat until the graph is empty.

**Complexity analysis.** Building the original  $G$  takes time  $O(\mathbf{SRC})$  for  $S$  BFSes, and the original matching takes time  $O((S+T)^3)$  (there are faster algorithms for matching, but Ford-Fulkerson is more widely known, simpler to code, and sufficient for this task). After that, we have at most  $T$  steps of altering the matching and removing some parts of it. Each of this steps requires building  $G'$ , which takes time  $O(\mathbf{SRC})$  for  $S$  BFSes, and after that, all steps are linear in the size of the graphs  $G$ ,  $G'$  or  $H$ , which are all bounded by  $O((S+T)^2)$ . Notice that building  $G'$  is by far the slowest step, so you can use less efficient implementations to manipulate the graphs without altering the time complexity. This gives an overall complexity of  $O(T\mathbf{SRC} + (S+T)^3)$ . This is enough to solve the problem, but it can be further refined by noticing each time we build  $G'$ , the BFSes will reach the same places or farther than in the previous step, so, if instead of restarting from scratch we remember the BFSes and continue from where we were stopped before, we can reduce the total time to build  $G'$  by a factor of  $T$ , down to  $O(\mathbf{SRC})$ , reducing the final complexity to  $O(\mathbf{SRC} + (S+T)^3)$ .